

Лабораторная работа №5

Хеш-функции SHA-1 и ГОСТ 34.11-2012 «Стрибог»

Пояснительная записка к защите. Реализация: *lab5/src*, язык *Rust*

Содержание

1. Что сделано в работе	2
2. Обозначения	2
3. Хеш-функции: общие понятия	2
3.1. Определение и требования	2
3.2. Итеративная схема	3
3.3. Атака удлинением	3
4. SHA-1	3
4.1. Паспорт алгоритма	3
4.2. Дополнение	3
4.3. Расширение блока	4
4.4. Раунд	4
4.5. Стойкость	4
5. ГОСТ 34.11-2012 «Стрибог»	5
5.1. Паспорт алгоритма	5
5.2. Преобразования S , P , L	5
5.3. Шифр E и расписание ключей	5
5.4. Функция сжатия	5
5.5. Этапы хеширования	6
5.6. Порядок байтов	6
5.7. Стойкость	6
6. Сравнение алгоритмов	6
7. Реализация	7
7.1. Где что лежит	7
7.2. Ошибки, найденные при отладке	7
8. Проверка работоспособности	7
9. Вопросы на защите	8

1. Что сделано в работе

Реализованы две криптографические хеш-функции: SHA-1 (160 бит) и ГОСТ 34.11-2012 «Стрибог» в двух вариантах, 256 и 512 бит. Всё написано с нуля, сторонних библиотек нет; меню и работа с файлами – общий модуль `common:cli`, как в предыдущих работах.

Модуль	Что реализует	Раздел
<code>hashing/sha1.rs</code>	дополнение, расширение блока до 80 слов, 80 раундов, выход 160 бит	4
<code>hashing/gost34_11.rs</code>	таблицы π , τ , A , C_i ; преобразования S , P , L , E , g_N ; этапы хеширования	5
<code>main.rs</code>	меню, обращение порядка байтов для ГОСТ, 10 контрольных примеров (команда <code>t</code>)	6, 7

Проверка: в меню команда `t` прогоняет контрольные примеры FIPS 180 / RFC 3174 (SHA-1) и RFC 6986 (ГОСТ), все 10 проходят.

2. Обозначения

Символ	Смысл
$M, M $	сообщение и его длина в битах
n	длина хеш-кода в битах: 160 для SHA-1, 256 или 512 для ГОСТ
$x \parallel y$	конкатенация
$x \oplus y$	поразрядное исключающее ИЛИ
$x \wedge y, x \vee y, \neg x$	поразрядные И, ИЛИ, НЕ
$\text{ROT}^k(x)$	циклический сдвиг 32-битного слова влево на k бит
$x + y$ (SHA-1)	сложение 32-битных слов по модулю 2^{32}
$x \boxplus y$ (ГОСТ)	сложение 512-битных векторов как чисел по модулю 2^{512}
$H_0 \dots H_4$	состояние SHA-1, пять 32-битных слов
h, N, Σ	состояние ГОСТ, счётчик обработанных бит и контрольная сумма блоков
$0^k, 1^k$	строка из k нулевых или единичных бит

3. Хеш-функции: общие понятия

3.1. Определение и требования

Хеш-функция H отображает сообщение произвольной длины в строку фиксированной длины n – хеш-код (дайджест). Ключа у неё нет, вычисляет её кто угодно. Криптографическая хеш-функция должна быть:

Свойство	Что нельзя сделать быстрее перебора	Перебор
стойкость к нахождению прообраза	по y найти M с $H(M) = y$	2^n
стойкость ко второму прообразу	по M найти $M' \neq M$ с $H(M') = H(M)$	2^n
стойкость к коллизиям	найти любую пару $M \neq M'$ с $H(M) = H(M')$	$2^{n/2}$

Кроме того, изменение одного бита сообщения должно менять в среднем половину бит хеш-кода (лавинный эффект), а хеш-код не должен выдавать никакой информации о сообщении.

Парадокс дней рождения

Среди q случайных значений из 2^n возможных пара совпадёт с вероятностью примерно $q^2/2^{n+1}$. Вероятность становится заметной при $q \approx 2^{n/2}$. Поэтому n -битная хеш-функция даёт не больше $n/2$ бит стойкости к коллизиям: у SHA-1 это 2^{80} , у ГОСТ-256 – 2^{128} , у ГОСТ-512 – 2^{256} .

3.2. Итеративная схема

Обе функции обрабатывают сообщение блоками по 512 бит:

$$h_0 = IV, \quad h_i = f(h_{i-1}, m_i), \quad H(M) = h_k,$$

где f – функция сжатия: из двух входов (состояние и блок) она делает один выход размера состояния. Это схема **Меркла – Дамгора**. Её главное свойство: если f стойка к коллизиям, а дополнение содержит длину сообщения (**MD-усиление**), то стойка и вся H .

Функции сжатия обычно строятся из блочного шифра $E_K(x)$:

Схема	Функция сжатия	Где
Дэвиса – Мейера	$f(h, m) = E_m(h) + h$ (блок сообщения – ключ)	SHA-1
Миягути – Пренеля	$f(h, m) = E_h(m) \oplus h \oplus m$ (состояние – ключ)	ГОСТ

Прибавление входа к выходу шифра («обратная связь») делает f необратимой: без неё по h_i и m_i можно было бы расшифровать h_{i-1} .

3.3. Атака удлинением

У чистой схемы Меркла – Дамгора хеш-код – это всё внутреннее состояние. Зная $H(M)$ и длину M , можно продолжить вычисление и получить $H(M \parallel \text{pad} \parallel X)$, не зная самого M . Поэтому конструкция $H(K \parallel M)$ не годится как имитовставка (для этого есть HMAC). SHA-1 этой атаке подвержен. В ГОСТ после блоков дополнительно обрабатываются N и Σ (раздел 5.5), и выход – уже не состояние после последнего блока.

4. SHA-1

4.1. Паспорт алгоритма

Параметр	Значение
Стандарт	FIPS 180-1 (1995), сейчас FIPS 180-4; описание также в RFC 3174
Блок	512 бит = 16 слов по 32 бита
Состояние и выход	160 бит = 5 слов
Раундов	80, по 20 на каждую из четырёх функций
Схема сжатия	Дэвиса – Мейера, операции ARX (сложение, сдвиг, логика)
Порядок байтов	big-endian: первый байт слова – старший
Статус	коллизия найдена в 2017 году (SHAttered), для подписей запрещён

4.2. Дополнение

$$M' = M \parallel 1 \parallel 0^k \parallel \llbracket M \rrbracket_{64}, \quad k \geq 0 \text{ минимальное с } |M'| \equiv 0 \pmod{512}.$$

То есть после сообщения ставится бит 1 (байт 80), затем нули до длины 448 по модулю 512, затем длина исходного сообщения в битах 64-битным big-endian числом. Дополнение добавляется всегда.

$ M $, байт	байт 80	нулей	Итог
0	1	55	1 блок

55	1	0	1 блок: 80 и длина помещаются впритык
56	1	63	2 блока: длине не хватает места
63	1	56	2 блока
64	1	55	2 блока

Число нулевых байт: $(56 - (|M|_{\text{байт}} + 1)) \bmod 64$, взятое неотрицательным. В Rust `%` – остаток со знаком, поэтому в коде `(56 + 64 - len % 64) % 64` после добавления 80.

4.3. Расширение блока

Блок разбивается на 16 слов $W_0 \dots W_{15}$ (big-endian), затем

$$W_t = \text{ROTL}^1(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}), \quad t = 16 \dots 79.$$

Сдвиг на 1 – единственное отличие SHA-1 от SHA-0 (1993): без него каждый бит W_t зависел лишь от одной позиции бит исходных слов, и SHA-0 был быстро взломан.

4.4. Раунд

$a, b, c, d, e := H_0, \dots, H_4$, затем для $t = 0 \dots 79$:

$$T = \text{ROTL}^5(a) + f_t(b, c, d) + e + K_t + W_t,$$

$$e := d, \quad d := c, \quad c := \text{ROTL}^{30}(b), \quad b := a, \quad a := T.$$

После 80 раундов: $H_i := H_i +$ соответствующая переменная. Выход – $H_0 \parallel \dots \parallel H_4$ big-endian.

Порядок присваиваний

T считается от **старых** $a \dots e$. Если сначала сдвинуть переменные, а потом вычислять a , получится другой (неверный) алгоритм. Поэтому в коде T – отдельная переменная.

Раунды	Функция	$f_t(b, c, d)$	K_t
0–19	Ch (выбор)	$(b \wedge c) \vee (\neg b \wedge d)$	5A827999
20–39	Parity	$b \oplus c \oplus d$	6ED9EBA1
40–59	Maj (большинство)	$(b \wedge c) \vee (b \wedge d) \vee (c \wedge d)$	8F1BBCDC
60–79	Parity	$b \oplus c \oplus d$	CA62C1D6

- **Ch**: бит b выбирает, чей бит взять – c (если 1) или d (если 0).
- **Maj**: результат – значение, которое в данной позиции имеют хотя бы два из трёх аргументов.
- Константы – «ничего в рукаве»: $K_t = \lfloor 2^{30} \sqrt{x} \rfloor$ для $x = 2, 3, 5, 10$. Начальное состояние 67452301 EFCDA89 98BADCFE 10325476 C3D2E1F0 – байты 01 23 45 67 89 AB CD EF, записанные в разном порядке, и C3D2E1F0.

4.5. Стойкость

Год	Результат
2005	Ван Сяюнь и др.: коллизия за $\approx 2^{69}$ операций вместо 2^{80}
2017	SHAttered (Google и CWI): первая практическая коллизия, два разных PDF с одним SHA-1, $\approx 2^{63}$ вычислений
2020	SHA-1 is a Shambles: коллизия с выбранным префиксом, подделка ключей PGP

Атаки на прообраз не найдены, поэтому SHA-1 ещё встречается там, где нужна только однонаправленность (например, HMAC-SHA1, идентификаторы объектов в git), но для подписей и сертификатов он запрещён.

5. ГОСТ 34.11-2012 «Стрибог»

5.1. Паспорт алгоритма

Параметр	Значение
Стандарт	ГОСТ Р 34.11-2012 (РФ), межгосударственный ГОСТ 34.11-2018, английское описание – RFC 6986
Блок, состояние	512 бит
Выход	512 бит или 256 бит (другой IV, старшая половина результата)
Функция сжатия	Миягути – Пренеля с 12-раундовым SPN-шифром E и счётчиком N
Порядок байтов	векторы записываются от старшего байта к младшему
Предшественник	ГОСТ Р 34.11-94 (на основе шифра ГОСТ 28147-89)

5.2. Преобразования S , P , L

Вектор $a = a_{63} \parallel \dots \parallel a_0$ из 64 байтов (a_0 – младший).

- $S(a) = \pi(a_{63}) \parallel \dots \parallel \pi(a_0)$ – нелинейная замена байтов. Таблица π – та же, что в шифре «Кузнечик» (ГОСТ 34.12-2015). Единственная нелинейная часть алгоритма.
- $P(a) = a_{\tau(63)} \parallel \dots \parallel a_{\tau(0)}$ – перестановка байтов $\tau = (0, 8, 16, \dots, 1, 9, \dots)$. Если записать 64 байта матрицей 8×8 по строкам, P её транспонирует. P – инволюция: $P(P(a)) = a$.
- $L(a) = l(a_{63..56}) \parallel \dots \parallel l(a_{7..0})$ – к каждому 64-битному слову отдельно применяется линейное отображение l : умножение на матрицу A размера 64×64 над $\text{GF}(2)$.

$$l(b_{63} \dots b_0) = \bigoplus_{i=0}^{63} b_i \cdot A_{63-i},$$

то есть XOR тех строк A , где у слова стоит единица, причём **старшему** биту b_{63} соответствует **первая** строка $A_0 = 8e20faa72ba0b470$, а b_0 – последняя $A_{63} = 641c314b2b8ee083$.

Зачем три преобразования

S даёт нелинейность (перемешивание), P разносит байты одного слова по разным словам, L перемешивает биты внутри слова (рассеивание). После P и L каждый байт выхода зависит от восьми байтов входа, после двух раундов – от всех 64. Это классическая SP-сеть, как в AES и «Кузнечике».

Проверка L : $\text{LPS}(0^{512}) = \text{b383fc2eced4a574}$, повторённое 8 раз (S даёт байты `fc`, P ничего не меняет, L одинаково для всех слов).

5.3. Шифр E и расписание ключей

$$X(K, a) = K \oplus a, \quad \text{LPS} = L \circ P \circ S,$$

$$E(K, m) = X(K_{13}, \text{LPS}(X(K_{12}, \dots \text{LPS}(X(K_1, m)) \dots))),$$

$$K_1 = K, \quad K_{i+1} = \text{LPS}(K_i \oplus C_i), \quad i = 1 \dots 12.$$

$C_1 \dots C_{12}$ – 512-битные константы стандарта. Ключи вычисляются **цепочкой** от предыдущего ключа, то есть расписание – это та же LPS-сеть, где вместо раундовых ключей используются константы.

5.4. Функция сжатия

$$g_N(h, m) = E(\text{LPS}(h \oplus N), m) \oplus h \oplus m.$$

Это схема Миягути – Пренеля: ключ шифра получается из состояния h , шифруется блок m , и к результату прибавляются оба входа. Счётчик N подмешивается в ключ, поэтому одинаковые блоки на разных позициях сжимаются по-разному.

5.5. Этапы хеширования

Этап	Действия
Этап 1	$h := IV: 0^{512}$ для 512 бит, байты 01 $((00000001)^{64})$ для 256 бит; $N := 0, \Sigma := 0$
Этап 2	пока $ M \geq 512$: m – младшие 512 бит M ; $h := g_N(h, m)$; $N := N \boxplus 512$; $\Sigma := \Sigma \boxplus m$; отрезать m от M
Этап 3	$m := 0^{511- M } \parallel 1 \parallel M$; $h := g_N(h, m)$; $N := N \boxplus M $; $\Sigma := \Sigma \boxplus m$; $h := g_0(h, N)$; $h := g_0(h, \Sigma)$
Выход	h для 512 бит, старшие 256 бит h – для 256 бит

Особенности по сравнению с SHA-1:

- Блоки берутся **с конца** записи сообщения (младшие биты числа) – следствие порядка записи, а не отдельное правило.
- Этап 3 выполняется **всегда**: при $|M| = 0$ (в том числе пустое сообщение или длина, кратная 512) последний блок равен $0^{511} \parallel 1$, а к N прибавляется 0.
- Дополнение – без длины; длина учитывается через N на завершении.
- Два дополнительных вызова g_0 с N и Σ делают выход зависящим от длины и суммы всех блоков и защищают от атаки удлинением.
- Сложение $\boxplus$ в N и Σ – по модулю 2^{512} , перенос идёт от младшего байта к старшему.

5.6. Порядок байтов

Самое частое место ошибок. Стандарт пишет векторы числами, старший байт слева. Байтовая строка (файл, текст) по соглашению читается наоборот: её первый байт – **младший** байт вектора, и дайджест печатается начиная с младшего байта.

	В записи RFC 6986	Как байтовая строка
M1	32313039...33323130 (число)	ASCII "0123...9012", 63 байта
ГОСТ-256(M1)	00557be5...ef1e159d	9d151eef...e57b5500
ГОСТ-512(M1)	486f64c1...1ad0541b	1b54d01a...c1646f48

В работе `gost34_11.rs` хранит векторы как в стандарте (индекс 0 – старший байт), поэтому таблицы A , C_i перенесены без изменений, а `main.rs` разворачивает сообщение перед хешированием и результат после. Так программа совпадает с распространёнными реализациями (проверено с Python-библиотекой `gostcrypto`). В таком представлении все операции должны быть согласованы: сложение по модулю 2^{512} идёт с индекса 63 к индексу 0, число N записывается в последние 8 байт, старшие 256 бит – это первые 32 байта.

5.7. Стойкость

Практических атак на полный «Стрибог» не известно; лучшие результаты касаются версий с уменьшенным числом раундов. Замечания, которые обсуждались в литературе: таблица π была опубликована без обоснования построения, а позже в ней нашли скрытую алгебраическую структуру (Бирюков, Перрен, Удовенко, 2016). Уязвимости это само по себе не даёт, но вызывает вопросы к процедуре выбора констант.

6. Сравнение алгоритмов

	SHA-1	ГОСТ 34.11-2012
Выход	160 бит	256 или 512 бит
Состояние	160 бит	512 бит (+ N и Σ)
Функция сжатия	Дэвиса – Мейера	Миягути – Пренеля
Шифр внутри	80 раундов ARX над 32-битными словами	12 раундов SPN над 512-битным вектором

Нелинейность	сложение по модулю 2^{32} , функции Ch и Maj	S-блок π
Дополнение	1, нули, 64-битная длина	нули, 1, без длины
Длина сообщения	в дополнении	в счётчике N и на завершении
Атака удлинением	подвержен	нет, благодаря $g_0(h, N)$ и $g_0(h, \Sigma)$
Коллизии	найжены (2017)	неизвестны
Скорость	быстрее: только 32-битные операции	медленнее без табличной оптимизации

7. Реализация

7.1. Где что лежит

- `sha1.rs`: `Shal::hash(&[u8]) -> Vec<u8>`. Дополнение, `from_be_bytes` для слов, `wrapping_add` для сложения по модулю 2^{32} , `rotate_left` для ROTL, диапазоны раундов – `match` с `0..20`, `20..40`, `40..60`, `60..80`.
- `gost34_11.rs`: константы `PI`, `A`, `TAU`, `C`; функции `s`, `p`, `l`, `key_schedule`, `e`, `g`, `add_mod_512`, `from_u64`, `final_hash`, публичные `hash_512` и `hash_256`.
- `main.rs`: меню, функции `gost_256` и `gost_512` (разворот байтов), таблица `TEST_VECTORS` и команда `t`.

Реализация прямолинейная, по формулам стандарта: L вычисляется перебором 64 бит каждого слова, а не по заранее вычисленным таблицам $S+L$, как в быстрых реализациях. На скорость это влияет, на результат – нет.

7.2. Ошибки, найденные при отладке

Полезно знать, так как на защите часто спрашивают «что будет, если...».

Где	Ошибка
SHA-1	переполнение <code>56 - len % 64</code> при <code>len % 64 > 56</code> ; длина в байтах вместо бит; little-endian вместо big-endian в словах, длине и выходе; диапазон <code>64..80</code> вместо <code>60..80</code> ; сдвиг переменных до вычисления T
ГОСТ, L	применение к 64 байтам сразу вместо восьми слов; A_i для бита i вместо A_{63-i}
ГОСТ, E	все ключи из K_1 вместо цепочки $K_{i+1} = \text{LPS}(K_i \oplus C_i)$
ГОСТ, g	ключ из $h \oplus m$ вместо $h \oplus N$
ГОСТ, этап 3	пропуск последнего блока при остатке 0 (пустое сообщение, длина кратна 64 байтам)
ГОСТ, константы	опечатки в трёх строках C_i при переписывании (11 байт)
ГОСТ, порядок байтов	часть функций в little-endian, часть в big-endian

Все эти ошибки выявляются контрольными примерами: любая из них меняет результат.

8. Проверка работоспособности

Алгоритм	Сообщение	Хеш-код (начало)
SHA-1	" "	DA39A3EE 5E6B4B0D ...
SHA-1	"abc"	A9993E36 4706816A ...
SHA-1	"abcbcbde...nopq", 56 байт	84983E44 1C3BD26E ...

SHA-1	миллион "a"	34AA973C D4C4DAA4 ...
ГОСТ-256	" "	3F539A21 3E97C802 ...
ГОСТ-256	M1	9D151EEF D8590B89 ...
ГОСТ-256	M2 (строка из «Слова о полку Игореве» в CP1251)	9DD2FE4E 90409E5D ...
ГОСТ-512	" "	8E945DA2 09AA869F ...
ГОСТ-512	M1	1B54D01A 4AF5B9D5 ...
ГОСТ-512	M2	1E88E622 26BFCA6F ...

Кроме того, ГОСТ сверен с `gostcrypto` на случайных сообщениях длиной 0, 1, 55, 56, 63, 64, 65, 127, 128, 129, 200 и 1000 байт, а SHA-1 файлов – с `shasum`. Лавинный эффект: замена ! на ? (4 бита) меняет 83 из 160 бит SHA-1 и 114 из 256 бит ГОСТ-256.

9. Вопросы на защите

? Что такое хеш-функция и чем она отличается от шифра?

Отображение сообщения произвольной длины в строку фиксированной длины. Ключа нет, функция необратима: по хеш-коду нельзя восстановить сообщение (прообразов бесконечно много). Шифр обратим при знании ключа и сохраняет длину данных.

? Чем хеш-функция отличается от имитовставки?

Имитовставка (MAC, как в ГОСТ 28147-89 в lab1) зависит от секретного ключа, и без ключа её не пересчитать. Хеш-код может пересчитать любой, поэтому он сам по себе защищает только от случайных искажений. Для защиты от подделки хеш-код подписывают или передают по защищённому каналу, либо используют HMAC.

? Перечислите требования к криптографической хеш-функции.

Стойкость к нахождению прообраза (2^n), второго прообраза (2^n) и коллизий ($2^{n/2}$), а также лавинный эффект. Из стойкости к коллизиям следует стойкость ко второму прообразу, обратное неверно.

? Почему стойкость к коллизиям – $2^{n/2}$, а не 2^n ?

Парадокс дней рождения: среди q случайных хеш-кодов есть совпадение с вероятностью около $q^2/2^{n+1}$, что становится заметным при $q \approx 2^{n/2}$. Для SHA-1 это 2^{80} , и поэтому 160 бит сегодня мало.

? Что такое схема Меркла – Дамгора и MD-усиление?

Сообщение дополняется и делится на блоки, состояние обновляется функцией сжатия: $h_i = f(h_{i-1}, m_i)$. Если в дополнение входит длина сообщения, то коллизия всей функции даёт коллизию f , то есть стойкость сводится к стойкости функции сжатия.

? Как дополняется сообщение в SHA-1? Сколько блоков у 56-байтного сообщения?

Байт 80, нули до 56 по модулю 64, 8 байт длины в битах, big-endian. У 56-байтного сообщения после 80 уже 57 байт, и для длины места нет, поэтому нужно 63 нулевых байта и второй блок: всего 128 байт.

? Зачем добавлять бит 1, если потом идут нули?

Чтобы дополнение было однозначным: иначе сообщения M и $M \parallel 0$ после дополнения нулями совпали бы. Единица отмечает конец данных. В SHA-1 от этого защищает ещё и длина, в ГОСТ длины в дополнении нет, и единица необходима.

? Что изменилось бы без ROTL¹ в расширении блока SHA-1?

Это был бы SHA-0. Без сдвига каждый бит W_t – XOR битов в той же позиции, биты разных позиций не перемешиваются, что позволило найти коллизии SHA-0 (1998 – теоретически, 2004 – практически).

? Откуда константы SHA-1?

$K_t = \lfloor 2^{30} \sqrt{x} \rfloor$ для $x = 2, 3, 5, 10$, начальное состояние – байты 0123456789ABCDEF и F0E1D2C3 в определённом порядке. Это «числа ничего в рукаве»: у автора не было свободы подобрать их под лазейку.

? Какие логические функции используются в SHA-1?

$\text{Ch} = (b \wedge c) \vee (\neg b \wedge d)$ – выбор c или d по биту b , раунды 0–19; $\text{Parity} = b \oplus c \oplus d$, раунды 20–39 и 60–79; Maj – большинство из трёх, раунды 40–59.

? Почему SHA-1 считается нестойким?

Для него найдены коллизии: в 2017 году SHAttered (два разных PDF с одним хешем, около 2^{63} вычислений), в 2020 году – коллизия с выбранным префиксом. Атак на прообраз нет, поэтому HMAC-SHA1 пока безопасен, но для подписей и сертификатов SHA-1 не используется.

? Что такое функция сжатия Дэвиса – Мейера и Миягути – Пренеля?

Способы получить функцию сжатия из блочного шифра. Дэвис – Мейер: $E_m(h) + h$, блок сообщения – ключ; так устроен SHA-1, где «шифр» – 80 раундов, а W_t – раундовые ключи. Миягути – Пренель: $E_h(m) \oplus h \oplus t$, состояние – ключ; так устроен «Стрибог». Прибавление входа нужно для необратимости.

? Из чего состоит раунд «Стрибога»?

X (XOR с раундовым ключом), затем S (замена байтов по π), P (транспонирование матрицы байтов 8×8) и L (умножение каждого 64-битного слова на матрицу A над $\text{GF}(2)$). 12 раундов и завершающее наложение ключа K_{13} .

? Как вычисляется L и какой строке A соответствует бит?

Для каждого из восьми 64-битных слов: XOR строк A , где в слове стоит 1. Старшему биту b_{63} соответствует A_0 , младшему b_0 – A_{63} . Проверка: $\text{LPS}(0) = \text{b383fc2eced4a574}$ восемь раз.

? Как вычисляются раундовые ключи в «Стрибоге»?

$K_1 = \text{LPS}(h \oplus N)$, $K_{i+1} = \text{LPS}(K_i \oplus C_i)$. Каждый ключ – из предыдущего, то есть расписание ключей – та же LPS-сеть, «зашифровывающая» константы C_i .

? Зачем в «Стрибоге» счётчик N и контрольная сумма Σ ?

N входит в ключ каждой функции сжатия, поэтому один и тот же блок на разных позициях обрабатывается по-разному (это мешает атакам на основе неподвижных точек и мультиколлизий). В конце обрабатываются N (общая длина) и Σ (сумма всех блоков), так что выход не совпадает с состоянием после последнего блока, и атака удлинением невозможна.

? Чем ГОСТ-256 отличается от ГОСТ-512?

Только начальным вектором ($(00000001)^{64}$ вместо 0^{512}) и тем, что берутся старшие 256 бит результата. Из-за разного IV ГОСТ-256 – это не обрезанный ГОСТ-512: старшая половина хеш-кода ГОСТ-512 с ним не совпадает.

? Что будет, если сообщение пустое или длина кратна 512 битам?

Этап 3 всё равно выполняется: последний блок – $0^{511} \parallel 1$, к N прибавляется 0. Если его пропустить, хеш-коды будут неверными (это была одна из ошибок при отладке).

? Почему в RFC 6986 хеш M1 равен `00557be5...`, а программа выводит `9d151eef...`?

Это один и тот же хеш-код в разном порядке байтов. Стандарт пишет вектор числом от старшего байта, а байтовая строка начинается с младшего. Для совместимости с другими реализациями программа разворачивает сообщение и результат.

? Как проверить правильность реализации?

Контрольными примерами стандартов (команда `tt`): FIPS 180 / RFC 3174 для SHA-1, RFC 6986 для ГОСТ. Отдельно полезны граничные длины 55, 56, 63, 64 и 65 байт (проверка дополнения) и сверка с независимой реализацией (`shasum`, `gostcrypto`).

? Где применяются хеш-функции?

Электронная подпись (подписывается хеш-код, а не документ), контроль целостности файлов, HMAC, хранение паролей (с солью и замедлением, как в PBKDF2), выработка ключей, блокчейн, системы контроля версий (git – SHA-1, с переходом на SHA-256).